

## Assignment 1: Imitation Learning

Due September 11, 11:59 pm

The goal of this assignment is to gain familiarity with imitation learning, including direct behavioral cloning and the DAGGER algorithm. In lieu of a human demonstrator, demonstrations will be provided via an expert policy that we have trained for you. Your goals will be to set up behavior cloning and DAGGER, and compare their performance on a few different continuous control tasks from the OpenAI Gym benchmark suite. Turn in your report and code as described in Section 6.

The starter-code for this assignment can be found at

[https://github.com/berkeleydeeprlcourse/homework\\_fall12023/tree/main/hw1](https://github.com/berkeleydeeprlcourse/homework_fall12023/tree/main/hw1)

You have the option of running the code either on Google Colab or on your own machine. Please refer to the README for more information on setup.

**Note:** The Colab is only used as a source of GPU compute, so you will be editing the same code regardless of what option you choose. For this assignment, since GPU will not be necessary, we strongly recommend running the code locally to gain some familiarity with installing the necessary packages. This will be extremely beneficial for later homeworks, so you can run experiments in parallel.

If you are running locally we *strongly* recommend you use Conda to manage your Python environment and dependencies. Instructions for installing Conda and setting up an environment are included.

## 1 Analysis

Consider the problem of imitation learning within a discrete MDP with horizon  $T$  and an expert policy  $\pi^*$ . We gather expert demonstrations from  $\pi^*$  and fit an imitation policy  $\pi_\theta$  to these trajectories so that

$$\mathbb{E}_{p_{\pi^*}(s)} \pi_\theta(a \neq \pi^*(s) \mid s) = \frac{1}{T} \sum_{t=1}^T \mathbb{E}_{p_{\pi^*}(s_t)} \pi_\theta(a_t \neq \pi^*(s_t) \mid s_t) \leq \varepsilon,$$

i.e., the expected likelihood that the learned policy  $\pi_\theta$  disagrees with the expert  $\pi^*$  within the training distribution  $p_{\pi^*}$  of states drawn from random expert trajectories is at most  $\varepsilon$ .

For convenience, the notation  $p_\pi(s_t)$  indicates the state distribution under  $\pi$  at time step  $t$  while  $p(s)$  indicates the state marginal of  $\pi$  across time steps, unless indicated otherwise.

1. Show that  $\sum_{s_t} |p_{\pi_\theta}(s_t) - p_{\pi^*}(s_t)| \leq 2T\varepsilon$ .

**Hint 1:** in lecture, we showed a similar inequality under the stronger assumption  $\pi_\theta(s_t \neq \pi^*(s_t) \mid s_t) \leq \varepsilon$  for every  $s_t \in \text{supp}(p_{\pi^*})$ . Try converting the inequality above into an expectation over  $p_{\pi^*}$ .

**Hint 2:** use the union bound inequality: for a set of events  $E_i$ ,  $\Pr[\bigcup_i E_i] \leq \sum_i \Pr[E_i]$

2. Consider the expected return of the learned policy  $\pi_\theta$  for a state-dependent reward  $r(s_t)$ , where we assume the reward is bounded with  $|r(s_t)| \leq R_{\max}$ :

$$J(\pi) = \sum_{t=1}^T \mathbb{E}_{p_\pi(s_t)} r(s_t).$$

- (a) Show that  $J(\pi^*) - J(\pi_\theta) = \mathcal{O}(T\varepsilon)$  when the reward only depends on the last state, i.e.,  $r(s_t) = 0$  for all  $t < T$ .
- (b) Show that  $J(\pi^*) - J(\pi_\theta) = \mathcal{O}(T^2\varepsilon)$  for an arbitrary reward.

## 作业1: 模仿学习

截止日期: 9月11日, 晚上11:59

本次作业的目标是熟悉模仿学习, 包括直接行为克隆和DAGger算法。由于没有人类演示者, 我们将通过我们为您训练的专家策略提供演示。您的目标将是设置行为克隆和DAGger, 并在OpenAI Gym基准套件的几个不同的连续控制任务上比较它们的性能。按照第6节的要求提交您的报告和代码。

本次作业的启动代码可以在以下位置找到

[https://github.com/berkeleydeeprlcourse/homework\\_fall12023/tree/main/hw1](https://github.com/berkeleydeeprlcourse/homework_fall12023/tree/main/hw1)

您可以选择在 Google Colab 上运行代码, 也可以在自己的机器上运行。有关设置的更多信息, 请参阅 README。

注意: Colab 仅用作 GPU 计算的来源, 因此无论您选择哪个选项, 都需要编辑相同的代码。对于这个作业, 由于不需要 GPU, 我们强烈建议在本地运行代码, 以熟悉安装必要包的过程。这对后续的作业将非常有帮助, 这样您就可以并行运行实验。

如果您在本地运行, 我们强烈建议您使用 Conda 来管理您的 Python 环境和依赖项。安装 Conda 和设置环境的说明已包含在内。

## 1 分析

考虑在具有  $T$  阶段和专家策略  $\pi^*$  的离散 MDP 中的模仿学习问题。我们从  $\pi^*$  收集专家演示, 并将模仿策略  $\pi_\theta$  拟合到这些轨迹上, 以便

$$\mathbb{E}_{p_{\pi^*}(s)} \pi_\theta(a \neq \pi^*(s) \mid s) = \frac{1}{T} \sum_{t=1}^T \mathbb{E}_{p_{\pi^*}(s_t)} \pi_\theta(a_t \neq \pi^*(s_t) \mid s_t) \leq \varepsilon,$$

即, 在从随机专家轨迹中抽取的状态训练分布  $p_{\pi^*}$  中, 学习到的策略  $\pi_\theta$  与专家  $\pi^*$  的预期不一致的可能性最多为  $\varepsilon$ 。

为方便起见,  $p_\pi(s_t)$  表示在  $\pi$  下的状态分布, 而  $p(s)$  表示  $\pi$  随时间步长的状态边缘, 除非另有说明。

1. 证明  $\sum_{s_t} |p_{\pi_\theta}(s_t) - p_{\pi^*}(s_t)| \leq 2T\varepsilon$ .

提示 1: 在讲座中, 我们展示了在更强的假设  $\pi_\theta(s_t \neq \pi^*(s_t) \mid s_t) \leq \varepsilon$  下类似的不等式

对每个  $s_t \in \text{supp}(p_{\pi^*})$ 。尝试将上述不等式转换为关于  $p_{\pi^*}$  的期望。

提示2: 使用并集界不等式: 对于一组事件  $E_i$   $\Pr[\bigcup_i E_i] \leq \sum_i \Pr[E_i]$ ,

2. 考虑在状态相关奖励  $r(s_t)$  下学习的策略  $\pi_\theta$  的预期回报, 其中我们假设奖励有界, 界限为  $|r(s_t)| \leq R_{\max}$ :

$$J(\pi) = \sum_{t=1}^T \mathbb{E}_{p_\pi(s_t)} r(s_t).$$

- (a) 证明  $J(\pi^*) - J(\pi_\theta) = \mathcal{O}(T\varepsilon)$ , 当奖励仅取决于最后一个状态, 即  $r(s_t) = 0$  时对所有  $t < T$  适用。

- (b) 证明  $J(\pi^*) - J(\pi_\theta) = \mathcal{O}(T^2\varepsilon)$  对于任意奖励都成立。

## 2 Editing Code

The starter code provides an expert policy for each of the MuJoCo tasks in OpenAI Gym. Fill in the blanks in the code marked with `TODO` to implement behavioral cloning. A command for running behavioral cloning is given in the README file.

We recommend that you read the files in the following order.

- `scripts/run_hw1.py` (training loop)
- `policies/MLP_policy.py` (policy definition)
- `infrastructure/replay_buffer.py` (stores training trajectories)
- `infrastructure/utils.py` (utilities for sampling trajectories from a policy)
- `infrastructure/pytorch_utils.py` (utilities for converting between NumPy/Pytorch)

For some files, some important functionality is missing and are marked with `TODO`. Specifically, you are asked to implement parts of the following:

- `policies/MLP_policy.py`: `forward` and `update` functions
- `infrastructure/utils.py`: `sample_trajectory` function
- `scripts/run_hw1.py`: `run_training_loop` function (most of your code will be in here)

## 3 Behavioral Cloning

1. Run behavioral cloning (BC) and report results on two tasks: one where a behavioral cloning agent should achieve at least 30% of the performance of the expert, and one environment of your choosing where it does not. Here is how you can run the Ant task:

```
python cs285/scripts/run_hw1.py \
  --expert_policy_file cs285/policies/experts/Ant.pkl \
  --env_name Ant-v4 --exp_name bc_ant --n_iter 1 \
  --expert_data cs285/expert_data/expert_data_Ant-v4.pkl \
  --video_log_freq -1
```

When providing results, report the mean and standard deviation of your policy’s return over multiple rollouts in a table, and state which task was used. When comparing one that is working versus one that is not working, be sure to set up a fair comparison in terms of network size, amount of data, and number of training iterations. Provide these details (and any others you feel are appropriate) in the table caption.

**Note:** What “report the mean and standard deviation” means is that your `eval_batch_size` should be greater than `ep_len`, such that you’re collecting multiple rollouts when evaluating the performance of your trained policy. For example, if `ep_len` is 1000 and `eval_batch_size` is 5000, then you’ll be collecting approximately 5 trajectories (maybe more if any of them terminate early), and the logged `Eval_AverageReturn` and `Eval_StdReturn` represents the mean/std of your policy over these 5 rollouts. Make sure you include these parameters in the table caption as well.

**Tip:** To generate videos of the policy, remove the flag `-video_log_freq -1` However, this is slower, and so you probably want to keep this flag on while debugging.

2. Experiment with one set of hyperparameters that affects the performance of the behavioral cloning agent, such as the amount of training steps, the amount of expert data provided, or something that you come up with yourself. For one of the tasks used in the previous question, show a graph of how the BC agent’s performance varies with the value of this hyperparameter. In the caption for the graph, state the hyperparameter and a brief rationale for why you chose it.

## 2 编辑代码

启动代码为 OpenAI Gym 中的每个 MuJoCo 任务提供了一个专家策略。填写 `TODO` 标记的代码中的空白以实现行为克隆。README 文件中给出了运行行为克隆的命令。

我们建议您按以下顺序阅读文件。

- `scripts/run_hw1.py` (训练循环)
- `policies/MLP_policy.py` (策略定义)
- `infrastructure/replay_buffer.py` (存储训练轨迹)
- `infrastructure/utils.py` (用于从策略中采样轨迹的工具)
- `infrastructure/pytorch_utils.py` (用于在 NumPy/Pytorch 之间转换的工具)

对于某些文件，一些重要的功能缺失，并用 `TODO` 标记。具体来说，要求你实现以下部分：

- `policies/MLP_policy.py`: 前向传播和更新函数
- `infrastructure/utils.py`: 采样\_轨迹函数
- `scripts/run_hw1.py`: 运行\_训练\_循环函数（你大部分代码将在这里）

## 3 行为克隆

1. 运行行为克隆（BC）并在两个任务上报告结果：一个任务是行为克隆代理应至少达到专家30%的性能，另一个是你选择的环境，其中它无法达到。以下是运行 Ant 任务的步骤：

```
python cs285/scripts/run_hw1.py \ --expert_policy_file
```

提供结果时，在表格中报告你的策略在多次运行中的平均回报和标准差，并说明使用的是哪个任务。在比较一个有效和一个无效的模型时，务必在网络大小、数据量和训练迭代次数方面设置公平的比较。在表格标题中提供这些详细信息（以及你认为任何其他适当的细节）。

注意：所谓“报告均值和标准差”是指你的`eval_batch_size`应该大于`ep_len`，这样你在评估训练好的策略性能时才能收集多个 rollout。例如，如果 `ep_len` 是 1000，而 `eval_batch_size` 是 5000，那么你将收集大约 5 条轨迹（如果其中有任何一条提前终止，数量可能会更多），而记录的 `Eval_AverageReturn` 和 `Eval_StdReturn` 代表你的策略在这 5 次 rollout 中的均值/标准差。确保也将这些参数包含在表格标题中。

提示：要生成策略的视频，请移除标志 `-video_log_freq -1`。但是，这会变慢，因此在调试时您可能希望保留此标志。

2. 尝试一组影响行为克隆代理性能的超参数，例如训练步数、提供的专家数据量或你自己想到的其他参数。针对上一题中使用的其中一个任务，展示一个图表，说明 BC 代理的性能如何随该超参数值的变化而变化。在图表的标题中，说明该超参数以及你选择它的简要理由。

## 4 DAGGER

1. Using the same code, you should be able to run DAGGER by modifying the runtime parameters as follows:

```
python cs285/scripts/run_hw1.py \
  --expert_policy_file cs285/policies/experts/Ant.pkl \
  --env_name Ant-v4 --exp_name dagger_ant --n_iter 10 \
  --do_dagger --expert_data cs285/expert_data/expert_data_Ant-v4.pkl \
  --video_log_freq -1
```

2. Run DAGGER and report results on the two tasks you tested previously with behavioral cloning. Report your results in the form of a learning curve, plotting the number of DAGGER iterations vs. the policy’s mean return, with error bars to show the standard deviation. Include the performance of the expert policy and the behavioral cloning agent on the same plot (as horizontal lines that go across the plot). In the caption, state which task you used, and any details regarding network architecture, amount of data, etc. (as in the previous section).

## 5 Discussion

Please answer the following short questions so we can improve future assignments.

- How much time did you spend on each part of this assignment?
- Any additional feedback?

## 6 Turning it in

1. **Submitting the PDF.** Make a PDF report containing: Responses for parts 1.1 and 1.2, Table 1 for a table of results from part 3.1, Figure 1 for part 3.2, Figure 2 with results from question part 4.2, and responses for parts 5.1 and 5.2.

See the handout at

<http://rail.eecs.berkeley.edu/deeprlcourse/static/misc/viz.pdf>

for notes on how to generate plots.

2. **Submitting the code and experiment runs.** In order to turn in your code and experiment logs, create a folder that contains the following:

- A folder named `run_logs` with experiments for both the behavioral cloning (part 2, not part 3) exercise and the DAGGER exercise. Note that you can include multiple runs per exercise if you’d like, but you must include at least one run (of any task/environment) per exercise. These folders can be copied directly from the `cs285/data` folder into this new folder. **Important: Disable video logging for the runs that you submit, otherwise the files size will be too large! You can do this by setting the flag `-video_log_freq -1`**
- The `cs285` folder with all the `.py` files, with the same names and directory structure as the original homework repository. Also include the commands (with clear hyperparameters) that we need in order to run the code and produce the numbers that are in your figures/tables (e.g. `run “python run_hw1_behavior_cloning.py --ep_len 200”` to generate the numbers for Section 2 Question 2) in the form of a README file.

## 4 DAgger

1. 使用相同的代码，你应该能够通过修改以下运行时参数来运行DAgger:

```
python cs285/scripts/run_hw1.py \ --expert_policy_file
```

2. 运行DAgger，并报告你在之前使用行为克隆测试的两个任务上的结果。以学习曲线的形式报告你的结果，绘制DAgger迭代次数与策略平均回报的对比，并用误差线显示标准差。在同一张图上包含专家策略和行为克隆代理的性能（作为横跨图表的水平线）。在图注中，说明你使用的任务，以及关于网络架构、数据量等任何细节（如前一节所述）。

## 5 讨论

请回答以下简短问题，以便我们改进未来的作业。

- 你在这份作业的每个部分上花了多少时间？
- 任何其他反馈？

## 6 转换itin

1. 提交PDF。制作一个PDF报告，其中包含：1.1和1.2部分的回答，3.1部分结果的表格1，3.2部分的图1，4.2问题部分的图2以及5.1和5.2部分的回答。

参见

<http://rail.eecs.berkeley.edu/deeprlcourse/static/misc/viz.pdf>

关于如何生成图表的笔记。

2. **提交代码和实验迭代结果。**实验日志，创建一个包含以下内容的文件夹：

- 一个名为 `run_logs` 的文件夹，其中包含行为克隆（第 2 部分，不是第 3 部分）练习和 DAgger 练习的实验。请注意，如果您愿意，每个练习可以包含多个运行，但每个练习至少必须包含一个运行（任何任务/环境）。这些文件夹可以直接从 `cs285/data` 文件夹复制到这个新文件夹。重要提示：为提交的运行禁用视频记录，否则文件大小会过大！您可以通过设置标志 `-video_log_freq -1` 来完成此操作。
- 包含所有 `.py` 文件的 `cs285` 文件夹，其名称和目录结构与原始作业存储库相同。还包含我们需要（带有清晰的超参数）的命令  
以便运行代码并生成您图表/表格中的数字（例如，运行 “python run\_hw1\_behavior\_cloning.py --ep\_len 200” 以生成第 2 节问题 2 的数字），并以 README 文件的形式提供。

As an example, the unzipped version of your submission should result in the following file structure. **Make sure that the submit.zip file is below 15MB and that they include the prefix q1\_ and q2\_.**



- 3. If you are a Mac user, **do not use the default “Compress” option to create the zip**. It creates artifacts that the autograder does not like. You may use `zip -vr submit.zip submit -x ".*DS_Store"` from your terminal.
- 4. Turn in your assignment on Gradescope. Upload the zip file with your code and log files to **HW1 Code**, and upload the PDF of your report to **HW1**.

例如，你的提交的解压版本应产生以下文件结构。确保 submit.zip 文件小于 15MB，并且它们包含前缀 q1\_ 和 q2\_。



- 3. 如果你是 Mac 用户，不要使用默认的“压缩”选项来创建 zip 文件。它会创建一些自动评分器不喜欢的无效文件。你可以从终端使用 `zip -vr submit.zip submit -x ".*DS_Store"` 来创建。
- 4. 在 Gradescope 上提交你的作业。将包含代码和日志文件的 zip 文件上传到 HW1 代码部分，并将报告的 PDF 上传到 HW1。